

Sprint 12 Structured Notes: SQL Basics

1. Current Progress

You are now moving from **database setup** into **real SQL practice**.

The full learning path is:

Tools → Terminal → Bash → Bash Scripting → Git → PostgreSQL → SQL → Normalization, Security, Performance

So far, you completed these stages:

Sprint 1: Tools and VS Code

You learned about:

- code editors and IDEs
- VS Code
- workspace
- project folders
- Live Server
- basic shortcuts and extensions

Simple meaning: Sprint 1 prepared your coding environment.

Sprint 2: Terminal, Shell, and Command Line Basics

You learned about:

- terminal
- command line
- shell
- Bash
- command history
- auto-complete
- clearing and interrupting commands

Simple meaning: Sprint 2 taught you what happens when you type commands.

Sprint 3: Bash File and Folder Commands

You learned how to:

- move between folders

- create files and folders
- rename, copy, and delete files
- read files
- use flags and options

Simple meaning: Sprint 3 taught you how to control files and folders from the terminal.

Sprint 4: Bash Scripting Basics

You learned about:

- Bash scripts
- `.sh` files
- shebang
- interpreter paths
- variables
- arrays
- loops
- `do` and `done`
- variable interpolation

Simple meaning: Sprint 4 taught you how to save terminal commands into reusable script files.

Sprint 5 to Sprint 10: Git Foundation

You learned:

- Git setup
- local Git workflow
- GitHub repositories and remotes
- branches and pull requests
- forks and sync
- `.gitignore`
- merge conflicts
- safer Git recovery tools
- beginner CI/CD ideas

Simple meaning: Sprints 5 to 10 taught you how developers track, share, review, and protect project changes.

Sprint 11: Relational Databases and PostgreSQL Setup

You learned:

- what a relational database is
- table, row, and column
- record, attribute, and field
- schema

- primary key and foreign key basics
- relational vs non-relational databases
- PostgreSQL
- pgAdmin
- psql
- PostgreSQL setup basics
- how to connect to the default `postgres` database

Simple meaning: Sprint 11 gave you the database foundation and PostgreSQL setup.

Sprint 12: SQL Basics

Now you are starting your first real SQL writing sprint.

In Sprint 11, you mainly learned **what a database is** and **how to open PostgreSQL tools**. In Sprint 12, you now learn how to:

- create a database
- create tables
- choose data types
- insert rows
- query rows
- filter data using simple conditions

Simple meaning: Sprint 12 is the point where you stop only learning database theory and start using SQL directly.

2. What Sprint 12 Is About

Sprint 12 is about the basics of SQL.

Now you start learning:

- what SQL is
- how SQL statements work
- how to end statements correctly
- how to create a database
- how to connect to a database
- how to create a table
- how to choose basic data types
- how to insert data into a table
- how to read data using `SELECT`
- how to filter data using `WHERE`

Simple meaning: Sprint 12 teaches you how to create simple database data and ask the database for answers.

3. Main Goal of Sprint 12

The main goal of Sprint 12 is:

Create tables and query simple data.

By the end of Sprint 12, you should be able to:

- explain what SQL is
- explain the difference between SQL and PostgreSQL
- understand statements and queries
- understand why semicolons are used
- know that `psql` shortcut commands are different from normal SQL
- create a database
- connect to a database
- create a table
- use `snake_case` naming
- understand basic data type categories
- use `INTEGER`, `SMALLINT`, `BIGINT`, and `SERIAL`
- use `VARCHAR` and `TEXT`
- use `DATE`, `TIME`, `TIMESTAMP`, and `TIMESTAMP WITH TIME ZONE`
- use `BOOLEAN`
- use `INSERT INTO`
- use `VALUES`
- use `SELECT`
- understand `*` wildcard
- use `WHERE`
- use simple comparison operators such as `=` and `<`

Simple meaning: Sprint 12 teaches you how to build a simple table, store data in it, and read back the data you want.

4. Why Sprint 12 Is Important

Sprint 12 is important because SQL is one of the main ways developers work with data.

Later, you will use SQL when working with:

- PostgreSQL databases
- project data
- business records
- student systems
- inventory systems
- customer data

- product tables
- reporting and analysis

Before learning keys, relationships, joins, normalization, security, and performance, you first need to feel comfortable with:

- creating simple tables
- storing simple rows
- reading simple results
- filtering data correctly

Simple meaning: Sprint 12 gives you the first real hands-on SQL skills that later database topics depend on.

5. What Is SQL?

SQL means:

Structured Query Language

It is the language used to work with relational databases.

SQL can be used to:

- create databases
- create tables
- insert data
- find data
- filter data
- change data
- delete data
- sort data
- summarize data

Simple meaning: SQL is the language you use to talk to a relational database.

SQL / SEQUEL History

SQL was originally called **SEQUEL**. Today, it is usually called **SQL**.

Simple meaning: SEQUEL is the older name. SQL is the common name used now.

SQL vs PostgreSQL

Many beginners confuse these two.

- **SQL** = the language

- **PostgreSQL** = the database system that understands and runs SQL

Easy comparison:

- English = language
- Teacher = person who understands the language

In the same way:

- SQL = language
- PostgreSQL = database software that understands SQL

Simple meaning: SQL is the language. PostgreSQL is the software.

6. What Is a Statement? What Is a Query?

In SQL, you send instructions to the database. These instructions are often called:

- statements
- queries
- commands

Statement

A statement is an instruction sent to the database.

Example:

```
CREATE DATABASE my_database;
```

This tells the database to create a new database.

Query

A query usually means a request for data.

Example:

```
SELECT * FROM dogs;
```

This asks the database to show data from the `dogs` table.

Simple meaning: A statement is a database instruction. A query is often a database question.

7. Semicolons

SQL statements usually end with a semicolon:

```
;
```

Example:

```
SELECT * FROM dogs;
```

The semicolon tells PostgreSQL:

This command is finished. Run it now.

If you forget the semicolon, `psql` may think your statement is still incomplete.

Wrong:

```
SELECT * FROM dogs
```

Correct:

```
SELECT * FROM dogs;
```

Simple meaning: The semicolon marks the end of most SQL statements.

8. psql Shortcut Commands

Inside `psql`, some commands begin with a backslash. These are **psql shortcut commands**, not normal SQL statements.

Examples:

- `\c my_database`
- `\q`

These do **not** need semicolons.

Example

```
 c my_database
```

This connects you to the `my_database` database.

Simple meaning: Backslash commands belong to `psql`, not normal SQL.

9. Starting PostgreSQL with psql

A common way to open PostgreSQL in the terminal is:

```
psql -U postgres -d postgres
```

Meaning:

- `psql` = open the PostgreSQL terminal tool
- `-U postgres` = connect as user `postgres`
- `-d postgres` = connect to the database named `postgres`

When connected, you may see this prompt:

```
postgres=#
```

Simple meaning: You are now inside PostgreSQL and ready to type commands.

10. CREATE DATABASE

To create a new database, use:

```
CREATE DATABASE my_database;
```

Meaning:

- `CREATE DATABASE` = create a new database
- `my_database` = the database name

Simple meaning: This creates a new database container.

Why We Create a Database

A database is the main place where tables will live.

Example idea:

- one database for school data
- one database for inventory data
- one database for store data

Simple meaning: The database is the main storage area. Tables go inside it.

11. Connect to a Database

After creating a database, you often connect to it using:

```
mysql> use my_database
```

This means:

- connect to `my_database`

After connecting, the prompt may change to:

```
mysql> use my_database
mysql>
```

Simple meaning: Now your SQL work is happening inside `my_database`.

12. CREATE TABLE

To create a table, use `CREATE TABLE`.

Example:

```
CREATE TABLE dogs (  
  id SERIAL,  
  name VARCHAR(100),  
  age INTEGER  
);
```

Meaning:

- `CREATE TABLE dogs` = create a table named `dogs`
- `id SERIAL` = ID column with auto-numbering
- `name VARCHAR(100)` = name column with text up to 100 characters
- `age INTEGER` = age column with whole numbers

Simple meaning: `CREATE TABLE` builds the structure of a table.

13. snake_case Naming

A common SQL naming style is **snake_case**.

This means:

- use lowercase letters
- separate words with underscores `_`

Good examples:

- `my_database`
- `student_records`
- `product_sales`
- `customer_orders`

Avoid names like:

- `MyDatabase`
- `Student Records`
- `customerOrders`

Simple meaning: Use lowercase names with underscores to keep SQL names clean and consistent.

14. Data Types

A data type tells PostgreSQL what kind of value a column can store.

Example:

- numbers
- text
- dates
- times
- true/false values

Simple meaning: A data type tells the database what kind of data belongs in a column.

Data Type Categories

At a beginner level, common data type categories are:

- numeric
- date/time
- character/string
- Unicode
- binary
- miscellaneous

Simple meaning: Data types are grouped by the kind of information they store.

15. Numeric Data Types

INTEGER

Stores whole numbers.

Example:

```
age INTEGER
```

Use it for things like:

- age
- quantity
- units sold
- counts

Simple meaning: `INTEGER` stores normal whole numbers.

SMALLINT

Stores smaller whole numbers.

Example:

```
rating SMALLINT
```

Simple meaning: `SMALLINT` stores smaller whole numbers than normal `INTEGER`.

BIGINT

Stores very large whole numbers.

Example:

```
population BIGINT
```

Simple meaning: `BIGINT` stores very large whole numbers.

SERIAL

`SERIAL` is used for automatic numbering.

Example:

```
id SERIAL
```

This is often used for ID columns. If you insert rows, PostgreSQL can automatically create values like:

- 1
- 2
- 3
- 4

Simple meaning: `SERIAL` automatically creates increasing numbers, often for IDs.

Important beginner note: `SERIAL` is commonly used, but it is not the same idea as normal text or date types. It is mainly used to help create auto-numbering ID columns.

16. Character and String Types

VARCHAR

`VARCHAR` stores variable-length text with a maximum length.

Example:

```
name VARCHAR(100)
```

This means the text can be up to 100 characters long.

Simple meaning: `VARCHAR` stores text up to a chosen limit.

TEXT

`TEXT` stores text without a fixed small limit like `VARCHAR(100)`.

Example:

```
description TEXT
```

Use it for:

- notes
- long descriptions
- comments
- paragraphs

Simple meaning: `TEXT` stores longer text.

VARCHAR vs TEXT

- `VARCHAR(100)` = text with a maximum length
- `TEXT` = longer flexible text

Simple meaning: Use `VARCHAR` when you want a length limit. Use `TEXT` when you want flexible longer text.

17. Date and Time Types

DATE

Stores a date.

Example:

```
event_date DATE
```

Example value:

```
'2026-06-02'
```

Simple meaning: `DATE` stores the day, month, and year.

TIME

Stores time only.

Example:

```
start_time TIME
```

Example value:

```
'09:30:00'
```

Simple meaning: `TIME` stores only the time.

TIMESTAMP

Stores date and time together.

Example:

```
event_timestamp TIMESTAMP
```

Example value:

```
'2026-06-02 09:30:00'
```

Simple meaning: `TIMESTAMP` stores both date and time.

TIMESTAMP WITH TIME ZONE

Stores date and time with timezone handling.

Example:

```
event_timestamp TIMESTAMP WITH TIME ZONE
```

Simple meaning: Use this when timezone matters.

18. BOOLEAN

`BOOLEAN` stores true or false values.

Example:

```
is_active BOOLEAN
```

Possible values:

- `TRUE`
- `FALSE`

Simple meaning: `BOOLEAN` stores yes/no or true/false type values.

19. Example Tables from Sprint 12

Products Table

```
CREATE TABLE products (  
  id SERIAL,  
  name VARCHAR(255)  
);
```

Simple meaning: This creates a simple table named `products` with an ID and a product name.

Dogs Table

```
CREATE TABLE dogs (  
  id SERIAL,  
  name VARCHAR(100),  
  age INTEGER  
);
```

Simple meaning: This creates a `dogs` table with three columns: ID, name, and age.

20. INSERT INTO

To add rows to a table, use `INSERT INTO`.

Example:

```
INSERT INTO dogs (name, age)
VALUES ('Gino', 3);
```

Meaning:

- `INSERT INTO dogs` = add data into the `dogs` table
- `(name, age)` = these are the columns you are filling
- `VALUES ('Gino', 3)` = these are the values going into those columns

Simple meaning: `INSERT INTO` adds a row into a table.

21. VALUES

`VALUES` is the part that gives the actual row data.

Example:

```
VALUES ('Gino', 3)
```

This means:

- `Gino` goes into the `name` column
- `3` goes into the `age` column

Simple meaning: `VALUES` gives the real data you want to store.

22. Why Explicit Column Names Are Safer

This is safer:

```
INSERT INTO dogs (name, age)
VALUES ('Gino', 3);
```

Why? Because the table also has an `id` column. Since `id` uses `SERIAL`, PostgreSQL can create it automatically.

If you try this carelessly:


```
INSERT INTO dogs  
VALUES ('Gino', 3);
```

You may create a problem because the table has more columns than the values you gave.

Simple meaning: Writing the column names clearly tells PostgreSQL exactly where each value should go.

23. Single Quotes for Text Values

Text values are usually written inside single quotes.

Correct:

```
'Gino'
```

Wrong:

```
Gino
```

Correct insert:

```
INSERT INTO dogs (name, age)  
VALUES ('Gino', 3);
```

Simple meaning: Put text values inside single quotes.

24. Insert One Row

Example:

```
INSERT INTO dogs (name, age)  
VALUES ('Gino', 3);
```

This adds one row.

Simple meaning: One `VALUES (...)` block adds one row.

25. Insert Multiple Rows

You can also insert more than one row at once.

Example:

```
INSERT INTO dogs (name, age)
VALUES
  ('Gino', 3),
  ('Nora', 2),
  ('Max', 5);
```

This adds three rows.

Simple meaning: Multiple value groups let you insert multiple rows in one statement.

26. SELECT

`SELECT` is used to read data from a table.

Example:

```
SELECT * FROM dogs;
```

This asks PostgreSQL to show data from the `dogs` table.

Simple meaning: `SELECT` is used to get data back from the database.

27. The `*` Wildcard

In SQL, `*` means **all columns**.

Example:

```
SELECT * FROM dogs;
```

This means:

- show all columns

- from the `dogs` table

If the table has `id`, `name`, and `age`, the result will show all three.

Simple meaning: `*` means show every column.

28. SELECT Specific Columns

You do not always need every column. You can choose only the columns you want.

Example:

```
SELECT name, age
FROM dogs;
```

This shows only:

- `name`
- `age`

It does not show `id`.

Simple meaning: You can choose exactly which columns to display.

29. WHERE

`WHERE` is used to filter rows.

Example:

```
SELECT *
FROM dogs
WHERE age < 3;
```

This means:

- show all columns
- from `dogs`
- but only where age is less than 3

If the data is:

- Gino, 3
- Nora, 2
- Max, 5

Then the result will show only Nora.

Simple meaning: `WHERE` lets you show only rows that match a condition.

30. Comparison Operators

Sprint 12 focuses on simple comparison operators.

`=`

Means equal to.

Example:

```
SELECT *  
FROM dogs  
WHERE name = 'Gino';
```

This finds rows where the name is exactly `Gino`.

Simple meaning: `=` checks for an exact match.

`<`

Means less than.

Example:

```
SELECT *  
FROM dogs  
WHERE age < 3;
```

This finds rows where age is less than 3.

Simple meaning: `<` finds values smaller than another value.

31. Filter by Number

Example:

```
SELECT *  
FROM dogs  
WHERE age < 3;
```

This filters by number.

Simple meaning: You can use `WHERE` and comparison operators to filter numeric data.

32. Filter by Text

Example:

```
SELECT age  
FROM dogs  
WHERE name = 'Gino';
```

This means:

- show the `age` column
- from the `dogs` table
- where the `name` is `Gino`

Simple meaning: You can use `WHERE` to filter by text too.

33. Full Sprint 12 Practice Flow

Run this slowly inside `psql`.

Step 1: Create the database

```
CREATE DATABASE my_database;
```

Step 2: Connect to it

```
\c my_database
```

Step 3: Create the products table

```
CREATE TABLE products (  
  id SERIAL,  
  name VARCHAR(255)  
);
```

Step 4: Create the dogs table

```
CREATE TABLE dogs (  
  id SERIAL,  
  name VARCHAR(100),  
  age INTEGER  
);
```

Step 5: Insert one dog using explicit columns

```
INSERT INTO dogs (name, age)  
VALUES ('Gino', 3);
```

Step 6: Insert multiple dogs

```
INSERT INTO dogs (name, age)  
VALUES  
  ('Nora', 2),  
  ('Max', 5);
```

Step 7: Query all columns

```
SELECT * FROM dogs;
```

Step 8: Query selected columns

```
SELECT name, age FROM dogs;
```

Step 9: Filter by age

```
SELECT * FROM dogs WHERE age < 3;
```

Step 10: Filter by name

```
SELECT age FROM dogs WHERE name = 'Gino';
```

Simple meaning: This practice flow covers the full core of Sprint 12.

34. Tasks in Sprint 12

You should practice these tasks:

- create `my_database`
- connect to it using `\c`
- create `products` and `dogs` tables
- insert one dog with explicit columns
- insert multiple dogs
- query all columns
- query selected columns
- filter by age and name

Simple meaning: These are the practical actions that turn Sprint 12 theory into real SQL skill.

35. Sprint 12 Checkpoint

You pass Sprint 12 when you can:

- create a table
- insert at least three rows
- write two `SELECT` queries using `WHERE`

Checkpoint examples:

```
SELECT *  
FROM dogs  
WHERE age < 3;
```

```
SELECT age  
FROM dogs  
WHERE name = 'Gino';
```

Simple meaning: If you can create, insert, and filter data correctly, you are ready to move forward.

36. What You Should Understand After Sprint 12

After Sprint 12, you should understand:

- SQL is the language used to work with relational databases
- SQL was originally called SEQUEL
- SQL statements usually end with semicolons
- `psql` shortcut commands are not normal SQL
- `CREATE DATABASE` creates a database
- `\c` connects to a database inside `psql`
- `CREATE TABLE` builds table structure
- `snake_case` is a good naming style
- data types control what each column can store
- `INTEGER`, `SMALLINT`, `BIGINT`, and `SERIAL` are numeric-related types
- `VARCHAR` and `TEXT` store text
- `DATE`, `TIME`, `TIMESTAMP`, and `TIMESTAMP WITH TIME ZONE` store date/time values
- `BOOLEAN` stores true/false values
- `INSERT INTO` adds rows
- `VALUES` contains the real data to store
- `SELECT` reads data
- `*` means all columns
- `WHERE` filters rows
- `=` means equal to
- `<` means less than

Simple meaning: After Sprint 12, you should be able to do simple real SQL work confidently.

37. Common Beginner Mistakes

Mistake 1: Forgetting the semicolon

Wrong:

```
SELECT * FROM dogs
```

Correct:

```
SELECT * FROM dogs;
```

Problem: PostgreSQL may think the statement is not finished.

Better habit: End normal SQL statements with `;`.

Mistake 2: Forgetting to connect to the new database

After creating a database, beginners sometimes forget to connect to it.

Correct command:

```
USE my_database
```

Problem: You may still be working in the wrong database.

Better habit: After creating the database, connect to it before creating tables.

Mistake 3: Not using single quotes for text

Wrong:

```
INSERT INTO dogs (name, age)
VALUES (Gino, 3);
```

Correct:

```
INSERT INTO dogs (name, age)
VALUES ('Gino', 3);
```

Problem: Text values need proper SQL string formatting.

Better habit: Put text values in single quotes.

Mistake 4: Inserting without clear column names

Risky:

```
INSERT INTO dogs
VALUES ('Gino', 3);
```

Better:

```
INSERT INTO dogs (name, age)
VALUES ('Gino', 3);
```

Problem: The table may have more columns than the values you gave.

Better habit: Explicitly mention column names.

Mistake 5: Confusing PostgreSQL and SQL

Wrong idea:

- SQL and PostgreSQL are the same thing

Correct idea:

- SQL = language
- PostgreSQL = database system

Better habit: Keep language and software separate in your mind.

Mistake 6: Thinking `*` and `WHERE` are the same idea

Wrong idea:

- `*` filters rows

Correct idea:

- `*` = all columns
- `WHERE` = filter rows

Better habit: Remember that columns and rows are different things.

38. Important Sprint 12 Commands and Syntax

```
psql -U postgres -d postgres
```

Connect to PostgreSQL using the `postgres` user and database.

```
postgres=#
```

The PostgreSQL prompt showing you are connected.

```
CREATE DATABASE my_database;
```

Create a new database.

```
\c my_database
```

Connect to `my_database`.

```
CREATE TABLE products (  
  id SERIAL,  
  name VARCHAR(255)  
);
```

Create a simple products table.

```
CREATE TABLE dogs (  
  id SERIAL,  
  name VARCHAR(100),  
  age INTEGER  
);
```

Create the dogs table.

```
INSERT INTO dogs (name, age)  
VALUES ('Gino', 3);
```

Insert one dog.

```
INSERT INTO dogs (name, age)  
VALUES  
  ('Nora', 2),  
  ('Max', 5);
```

Insert multiple dogs.

```
SELECT * FROM dogs;
```

Show all columns from `dogs`.

```
SELECT name, age FROM dogs;
```

Show selected columns.

```
SELECT * FROM dogs WHERE age < 3;
```

Filter by age.

```
SELECT age FROM dogs WHERE name = 'Gino';
```

Filter by name.

39. Mini Assessment Before Sprint 13

Answer these questions before moving forward:

1. What is SQL?
2. What does SEQUEL refer to in SQL history?
3. What is the difference between SQL and PostgreSQL?
4. What is a statement?
5. What is a query?
6. Why do SQL statements usually end with semicolons?
7. What makes a `psql` shortcut command different from normal SQL?
8. What does `CREATE DATABASE` do?
9. What does `\c my_database` do?
10. What does `CREATE TABLE` do?
11. Why is `snake_case` recommended?
12. What is a data type?
13. Name the basic data type categories mentioned in Sprint 12.
14. What does `INTEGER` store?
15. What does `SMALLINT` store?
16. What does `BIGINT` store?
17. What is `SERIAL` used for?
18. What is the difference between `VARCHAR` and `TEXT`?
19. What does `DATE` store?
20. What does `TIME` store?
21. What does `TIMESTAMP` store?
22. What does `TIMESTAMP WITH TIME ZONE` mean in simple words?
23. What does `BOOLEAN` store?
24. What does `INSERT INTO` do?

25. What does `VALUES` do?
 26. Why is it safer to mention column names in `INSERT`?
 27. What does `SELECT` do?
 28. What does `*` mean in `SELECT *`?
 29. What does `WHERE` do?
 30. What does `=` mean?
 31. What does `<` mean?
 32. Write one query that shows dogs younger than 3.
 33. Write one query that shows the age of the dog named Gino.
-

40. Sprint 12 Completion Checklist

Tick these when completed:

- ☐ I can explain what SQL means.
 - ☐ I know that SQL was originally called SEQUEL.
 - ☐ I know the difference between SQL and PostgreSQL.
 - ☐ I understand statements and queries.
 - ☐ I understand why semicolons are used.
 - ☐ I understand that `psql` backslash commands are not normal SQL statements.
 - ☐ I can open PostgreSQL using `psql`.
 - ☐ I can recognize the PostgreSQL prompt.
 - ☐ I can create a database using `CREATE DATABASE`.
 - ☐ I can connect to a database using `\c`.
 - ☐ I can create a table using `CREATE TABLE`.
 - ☐ I can use `snake_case` names.
 - ☐ I understand what a data type is.
 - ☐ I understand numeric types like `INTEGER`, `SMALLINT`, `BIGINT`, and `SERIAL`.
 - ☐ I understand text types like `VARCHAR` and `TEXT`.
 - ☐ I understand `DATE`, `TIME`, `TIMESTAMP`, and `TIMESTAMP WITH TIME ZONE`.
 - ☐ I understand `BOOLEAN`.
 - ☐ I can use `INSERT INTO`.
 - ☐ I can use `VALUES`.
 - ☐ I can insert one row correctly.
 - ☐ I can insert multiple rows correctly.
 - ☐ I can use `SELECT`.
 - ☐ I understand that `*` means all columns.
 - ☐ I can select only the columns I need.
 - ☐ I can use `WHERE` to filter rows.
 - ☐ I can use `=` for exact matches.
 - ☐ I can use `<` for smaller values.
 - ☐ I can create the `products` and `dogs` tables from Sprint 12.
 - ☐ I completed the Sprint 12 checkpoint task.
-

41. Final Simple Meaning of Sprint 12

Sprint 12 teaches you your first real SQL workflow:

open PostgreSQL → create a database → connect to it → create tables → insert rows → query rows → filter rows

If you can do that confidently, you are ready for Sprint 13: **Keys, Relationships, and Joins**.